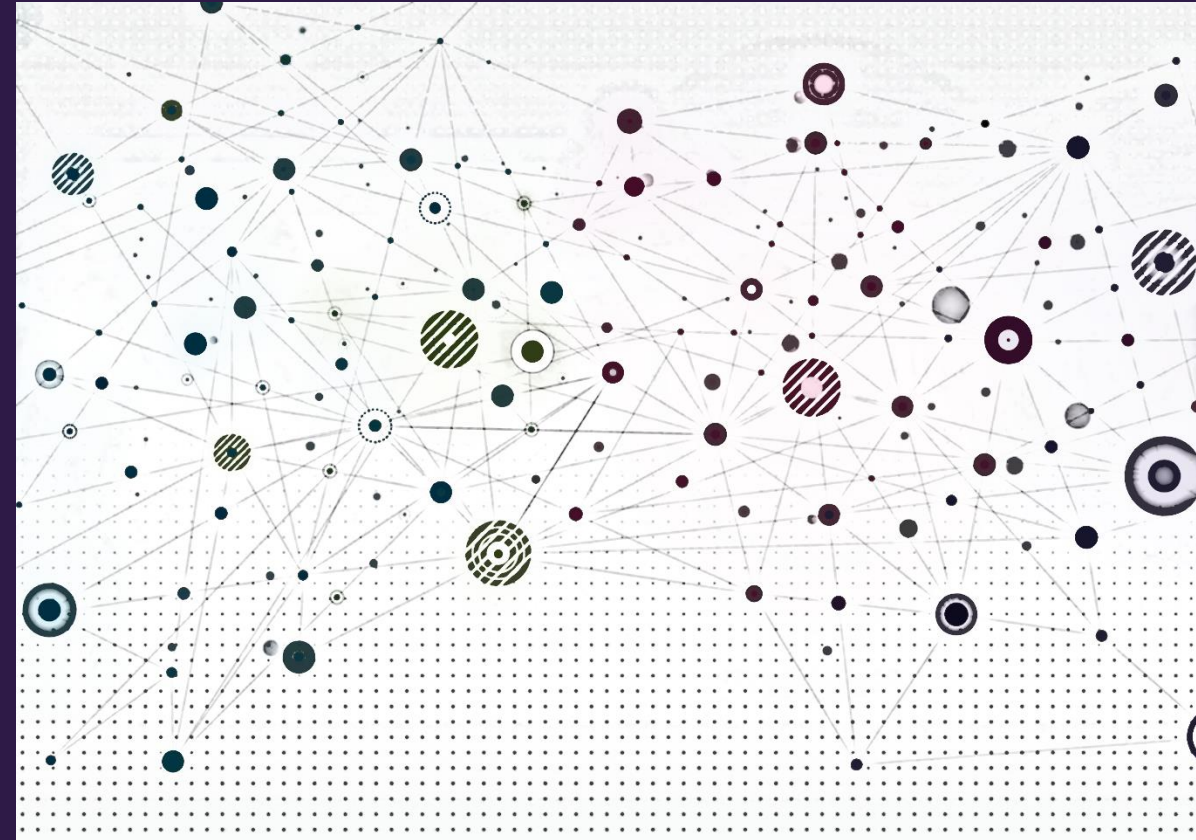


Knowledge graphs





Overview

- Recap
- Knowledge graphs
- Knowledge graph completion



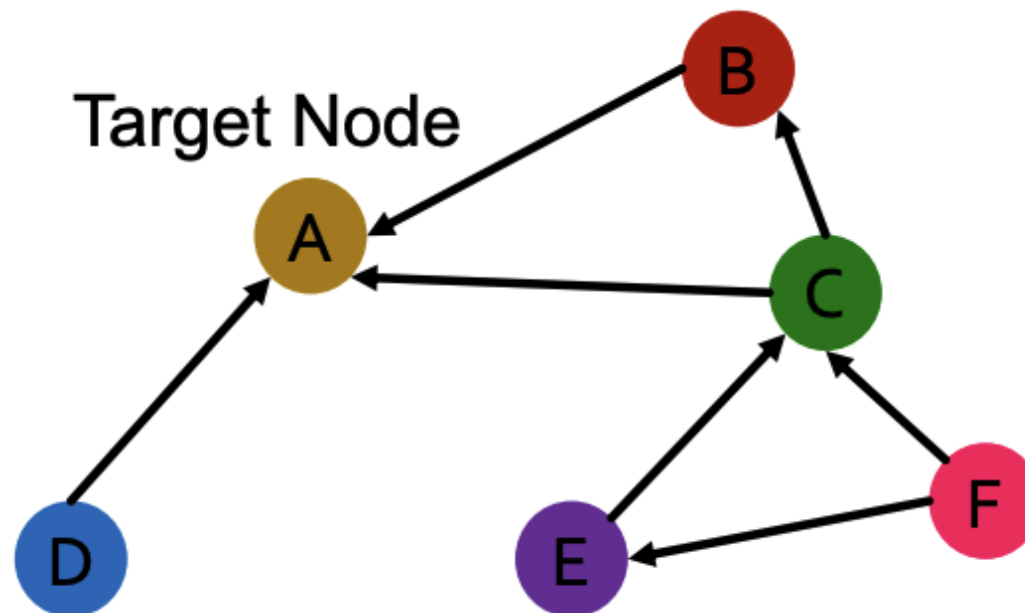
Recap





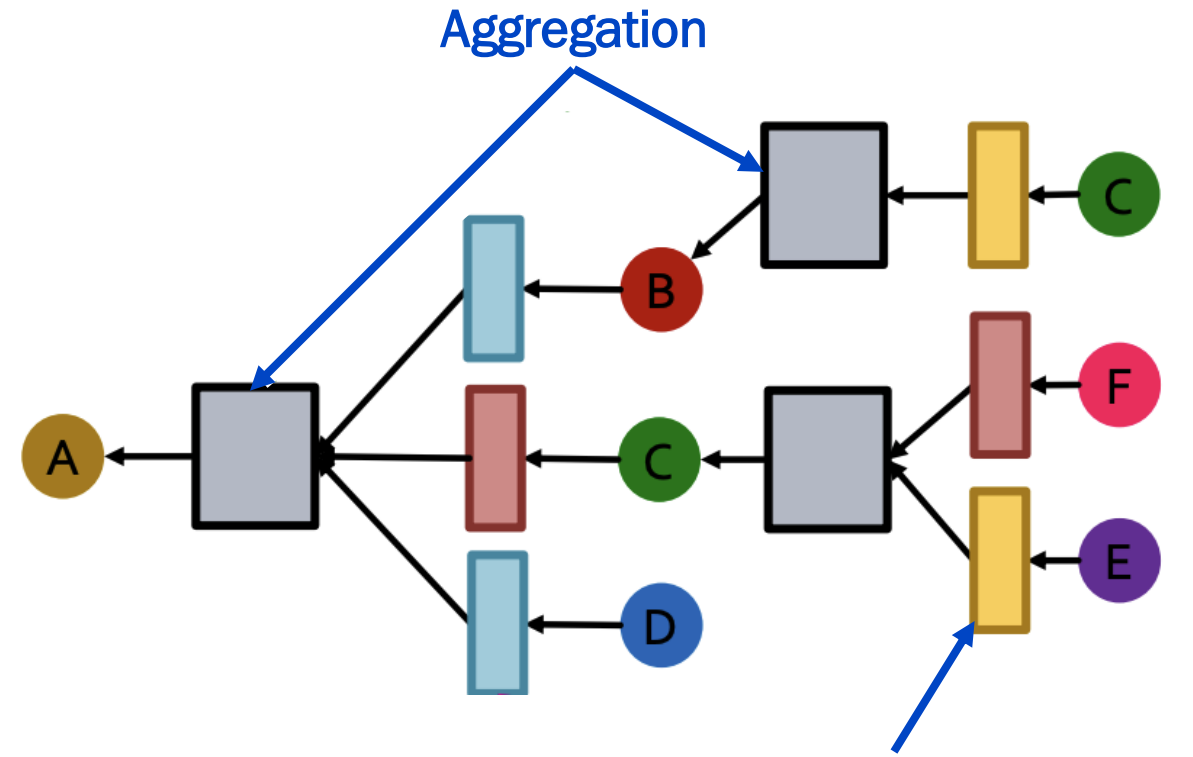
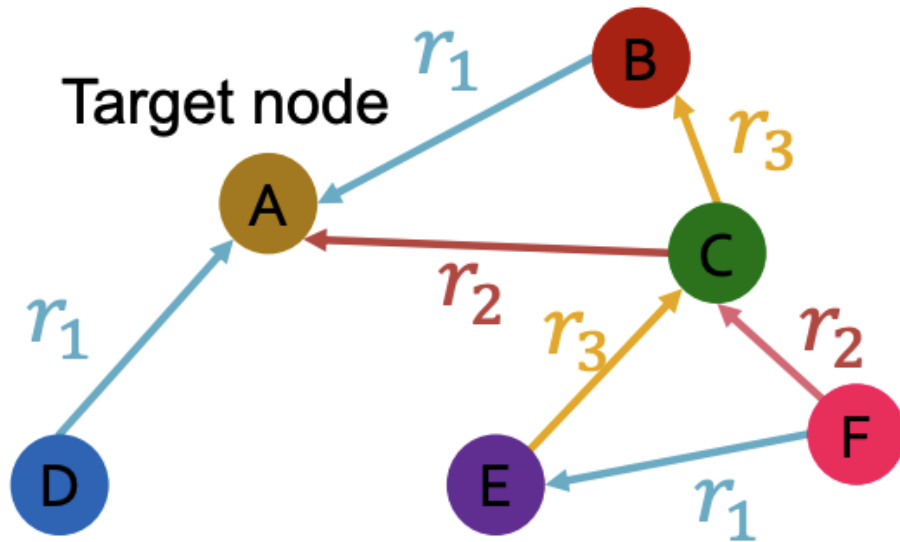
Heterogeneous graphs

- Multiple node and / or edge types yielding multiple *relation* types



Relational graph convolutional networks (RGCN)

- Extend GCN to handle heterogeneous graphs with multiple relation types
- Different neural networks for different relation types



Distinct neural network for each relation type



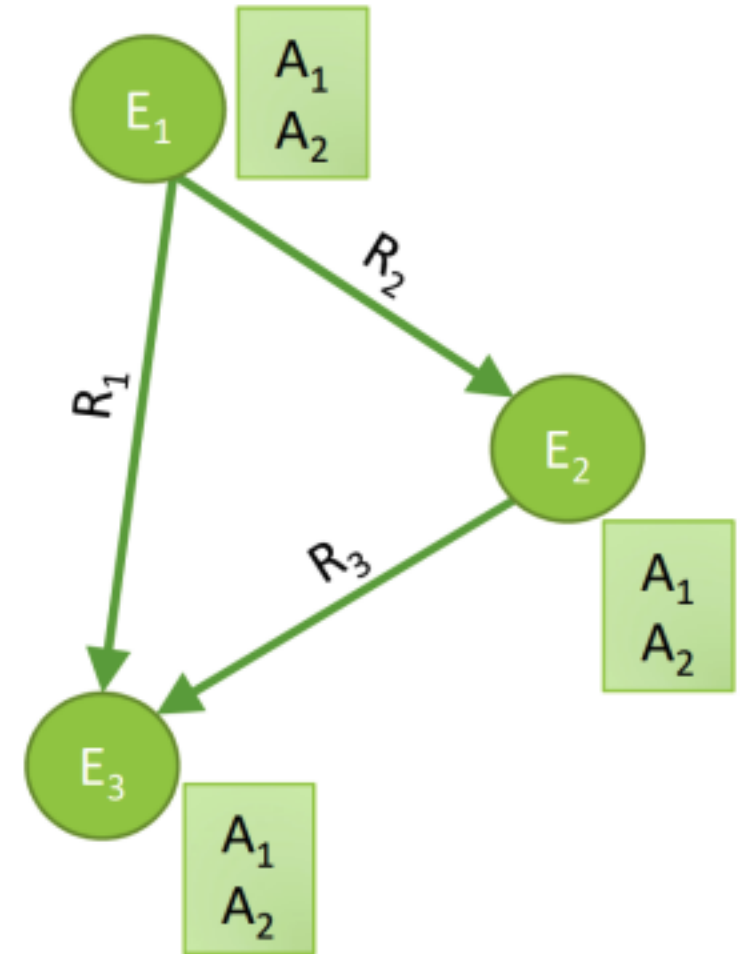
Knowledge graphs





Knowledge graphs

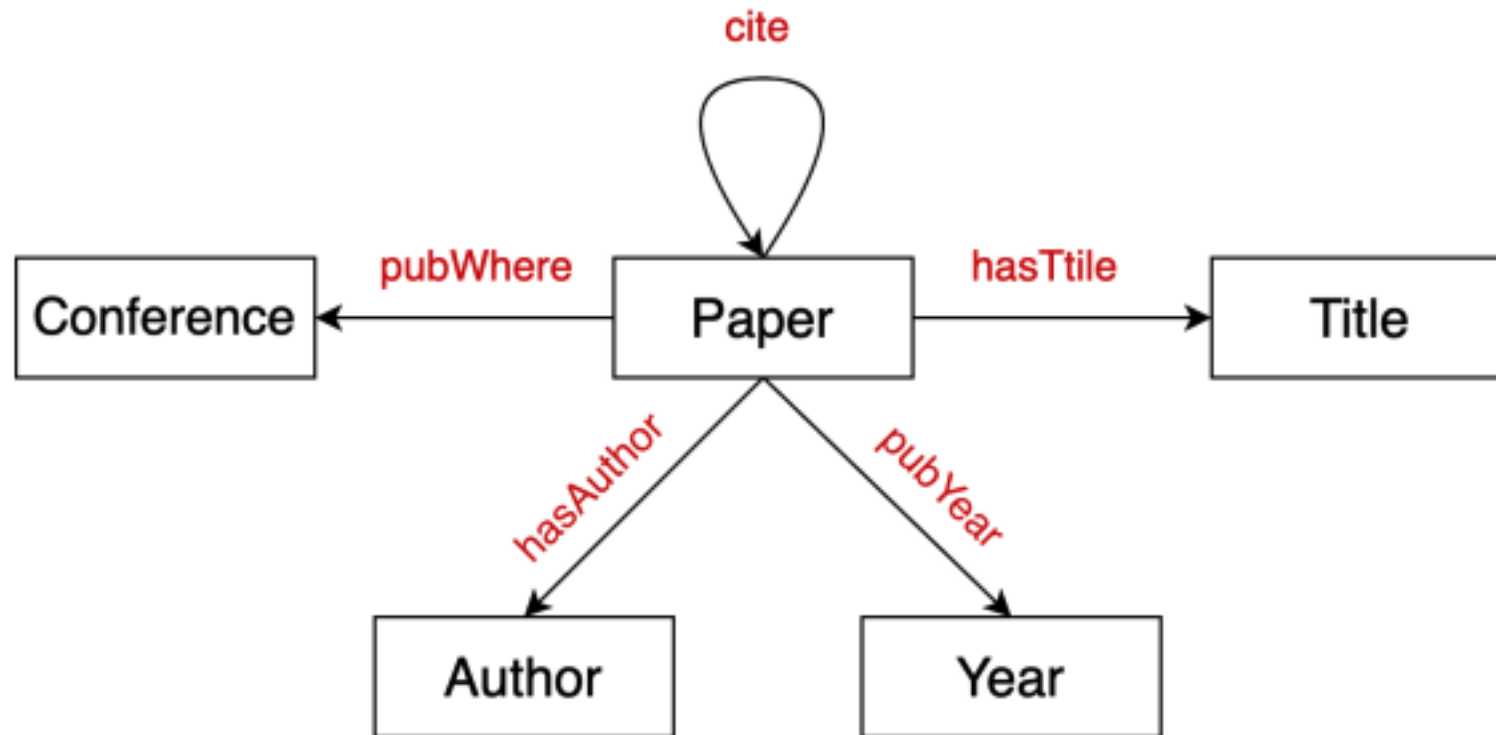
- Heterogeneous graphs that represent entities, types, and relationships
- Nodes are *entities*
- Nodes are labeled with their *types*
- Edges between nodes indicate specific *relationships* between entities, often *directional*





Bibliographic networks

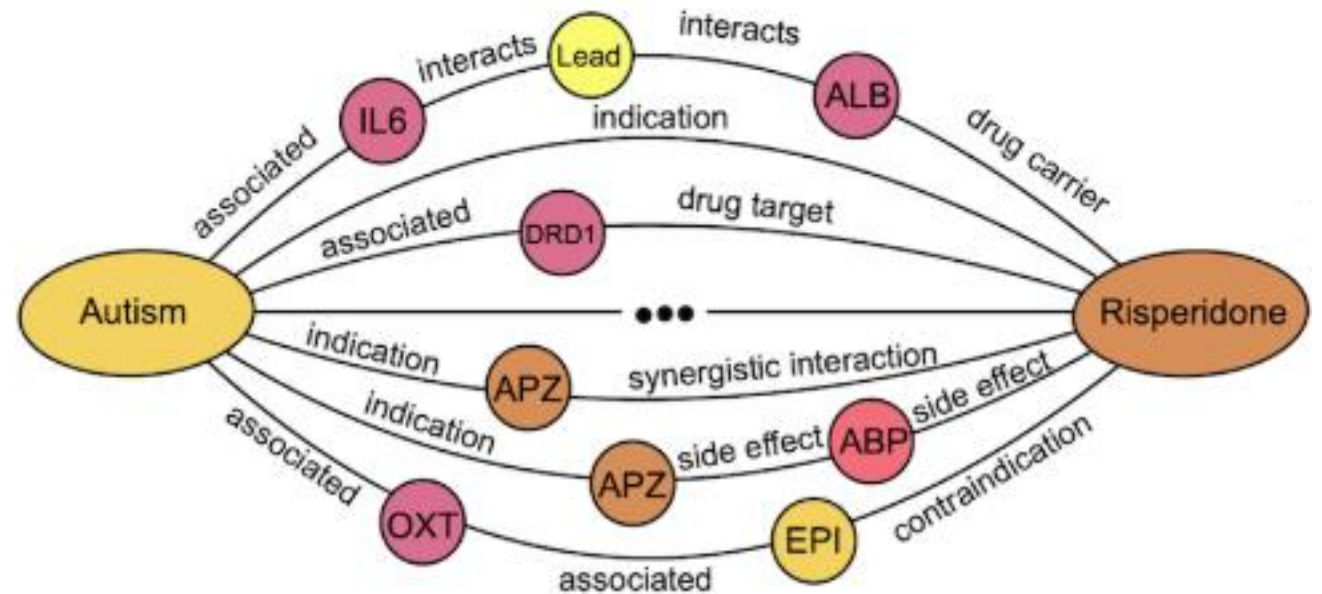
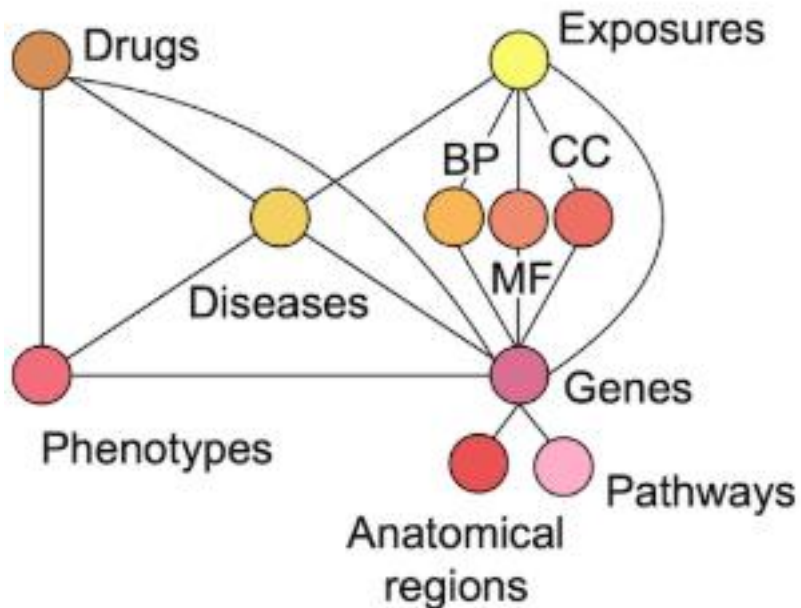
- Node types: paper, title, author, conference, year
- Relation types: pubWhere, pubYear, hasTitle, hasAuthor, cite





Biomedical knowledge graphs

- Node types: drugs, biological pathways, diseases, ...
- Relation types: associated, interacts, ...





More knowledge graphs

- Google knowledge graph - <https://developers.google.com/knowledge-graph>
- Amazon product graph
- Facebook graph API - <https://developers.facebook.com/docs/graph-api/>
- Microsoft graph - <https://learn.microsoft.com/en-us/graph/overview>
- LinkedIn knowledge graph

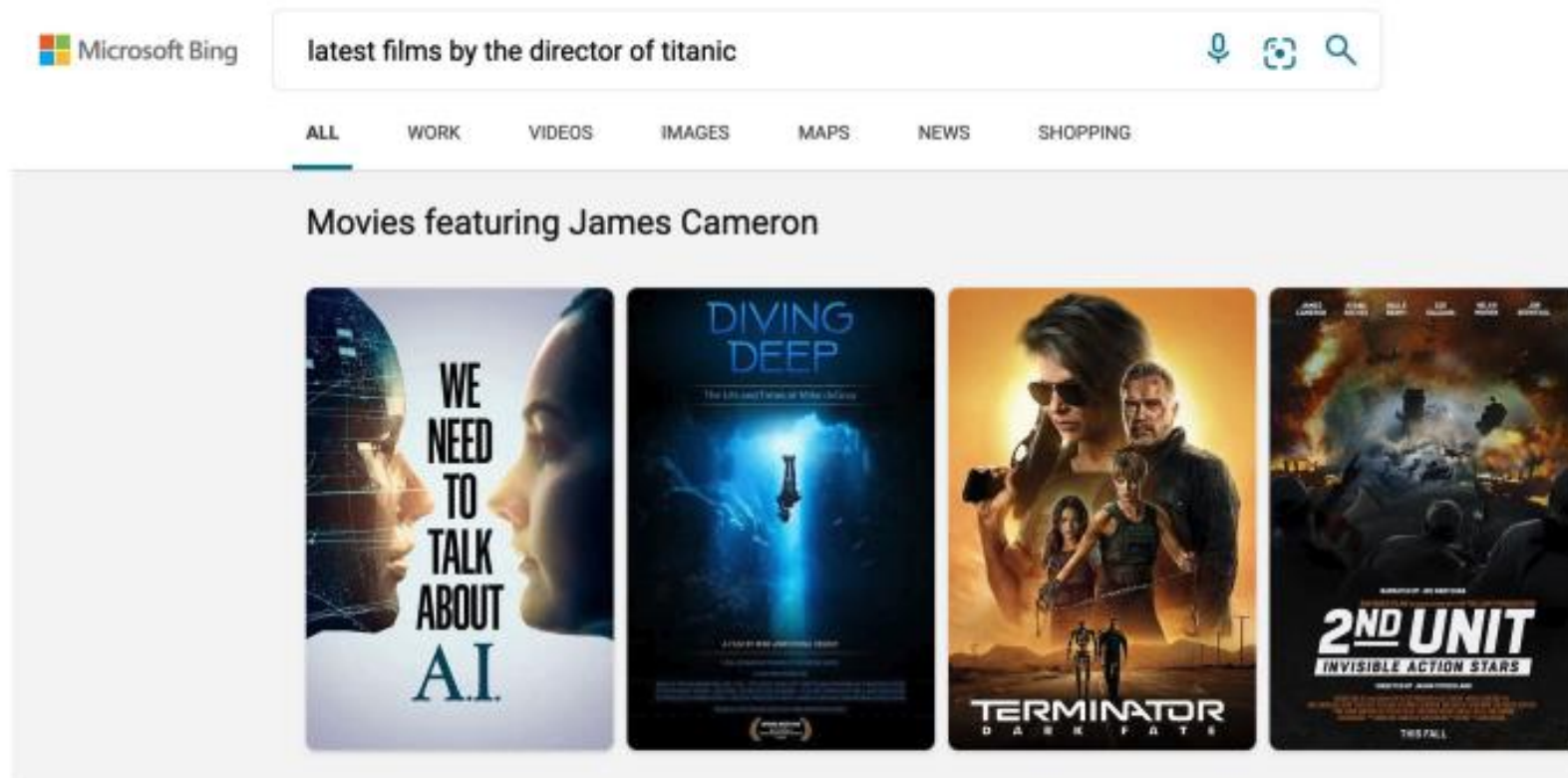


Publicly available knowledge graphs

- Wikidata - https://www.wikidata.org/wiki/Wikidata:Main_Page
- Dbpedia - <https://www.dbpedia.org/resources/knowledge-graphs/>
- YAGO - <https://yago-knowledge.org/>
- NELL - <http://rtw.ml.cmu.edu/rtw/kbbrowser/>

Applications of knowledge graphs

- Serving information
- Augmenting LLMs (we'll come back to this in a future lecture)

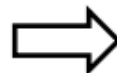




Common characteristics of modern knowledge graphs

- Massive: Millions (or Billions) of nodes and edges
 - Even scientific knowledge graphs have up to hundreds of thousands of nodes and edges
- Incomplete: Many true edges are missing (missing knowledge)

**Given a massive KG,
enumerating all the
possible facts is
intractable!**



**Can we predict plausible
BUT missing links?**



Knowledge graph completion

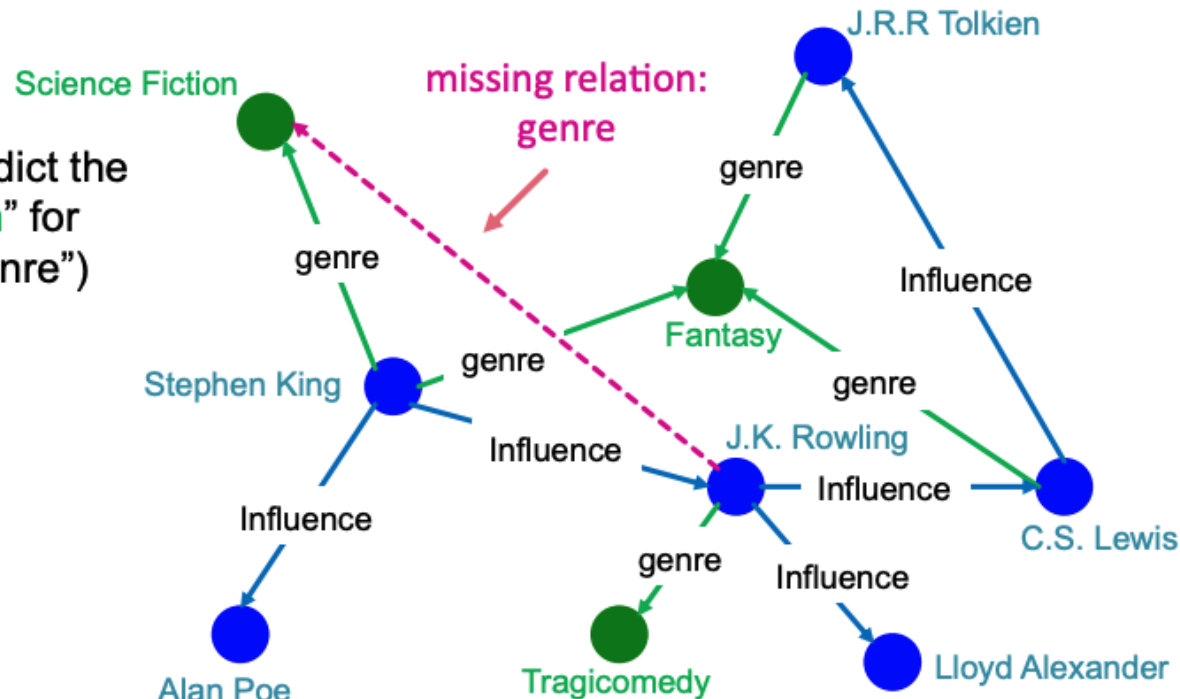




Knowledge graph (KG) completion task

- Given a large KG, can we "complete" it (i.e., add missing edges)
- Task is typically formulated as predicting tails (or consequents)
 - Consider relations in a directed KG represented as a tuple (head, relation, tail) where head (h) has relation (r) with tail (t)
 - Given (head, relation) predict missing tails

Example task: predict the tail "Science Fiction" for ("J.K. Rowling", "genre")

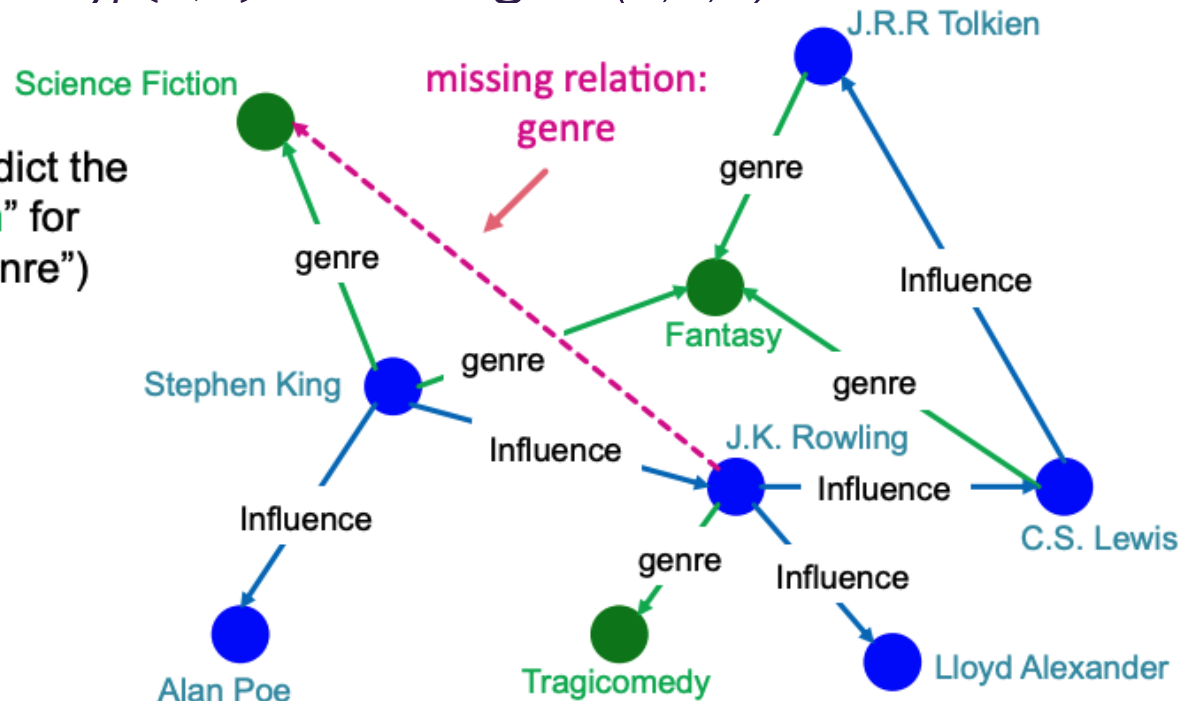




Knowledge graph (KG) completion task

- Model entities (nodes) and relations (edges) as embeddings in $\mathbb{R}^k$
- Given a triple (h, r, t) , the goal is to form an embedding of (h, r) that is close to the embedding of t
 - How to embed (h, r) ?
 - How to define a score $f_r(h, t)$ that is high if (h, r, t) exists and low otherwise

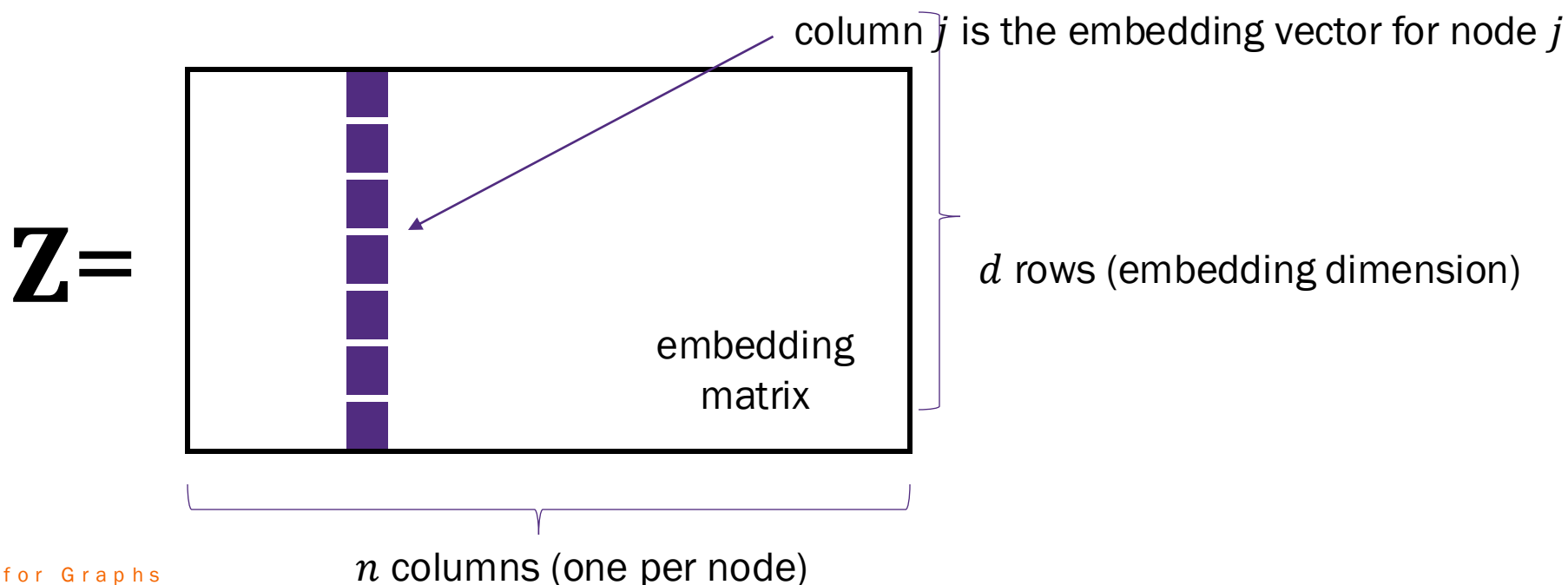
Example task: predict the tail “Science Fiction” for (“J.K. Rowling”, “genre”)





Transductive methods for KG completion

- “Standard” KG completion methods use **shallow embedding methods** for computational efficiency on large KGs
 - We’ll focus on some of these that are based on different geometric intuitions
- Recently (<5 years) inductive GNN methods have been developed
 - See <https://link.springer.com/article/10.1007/s00521-023-09286-2>





Knowledge graph (KG) completion task

- We want our approaches to handle logical properties
 - Symmetric (Antisymmetric) $r(h, t) \Rightarrow r(t, h)$, $r(h, t) \Rightarrow !r(h, t)$
 - Example: Family (symmetric), Parent (antisymmetric)
 - NOTE: A relation cannot be symmetric and antisymmetric
 - Inverse: $r_1(h, t) \Rightarrow r_2(t, h)$
 - Example: (Mentor, mentee)
 - Composition (Transitivity): $r_1(x, y) \wedge r_2(y, z) \Rightarrow r_3(x, z)$
 - Example: parent (Joe, Linda) $\wedge$ father (Linda, Steve) $\Rightarrow$ grandfather (Joe, Steve) (the father of my parent is my grandfather)
 - 1-to-N: $r(h, t_1), r(h, t_2), \dots, r(h, t_n)$ are all True
 - Example: r is “enrolled in ML4G”

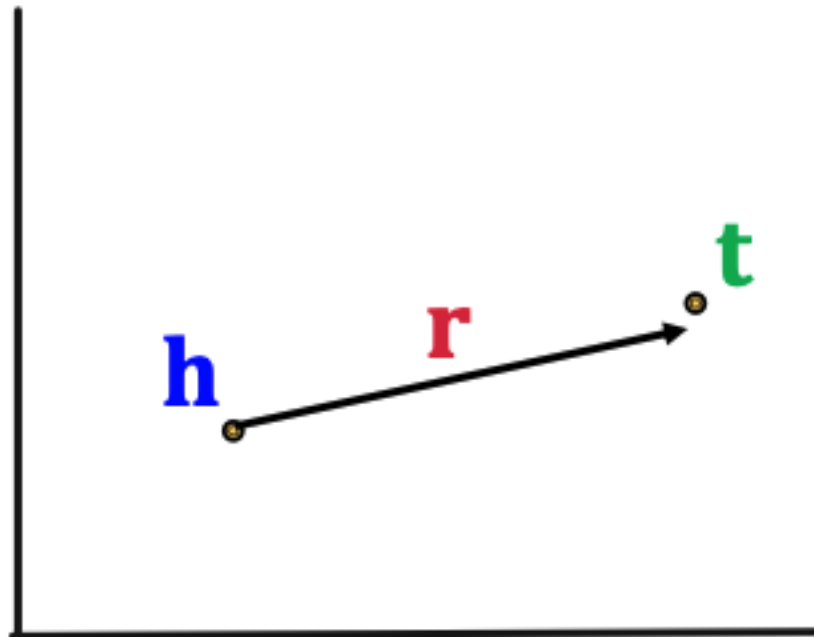


TransE method



Trans(lation) Embedding (TransE)

- Let $\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{R}^k$ be embeddings for the relation (h, r, t)
- **TransE**: $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ if the relation (h, r, t) exists, else $\mathbf{h} + \mathbf{r} \neq \mathbf{t}$
- Define the scoring function as $f_r(h, t) = -\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|$
- Geometric interpretation as vectors will help us examine the ability of **TransE** to model different logic relations





TransE learning algorithm

Algorithm 1 Learning TransE

input Training set $S = \{(h, r, t)\}$, entities and rel. sets E and R , margin γ , embeddings dim. k .

1: **initialize** $r \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each $r \in R$
 2: $r \leftarrow r / \|r\|$ for each $r \in R$
 3: $e \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each entity $e \in E$

Initialize relations r and entities e uniformly, then normalize.
 γ is the margin.

4: **loop**

5: $e \leftarrow e / \|e\|$ for each entity $e \in E$

6: $S_{batch} \leftarrow \text{sample}(S, b)$ // sample a minibatch of size b

7: $T_{batch} \leftarrow \emptyset$ // initialize the set of pairs of triplets

8: **for** $(h, r, t) \in S_{batch}$ **do**

9: $(h', r, t') \leftarrow \text{sample}(S'_{(h, r, t)})$ // sample a corrupted triplet

Sample triplet (h', r, t') that does not appear in the KG.

10: $T_{batch} \leftarrow T_{batch} \cup \{((h, r, t), (h', r, t'))\}$

11: **end for**

12: Update embeddings w.r.t.

$$\sum_{((h, r, t), (h', r, t')) \in T_{batch}} \nabla [\gamma + \underset{\text{positive sample}}{d(h + r, t)} - \underset{\text{negative sample}}{d(h' + r, t')}]_+$$

d represents distance (negative of score)

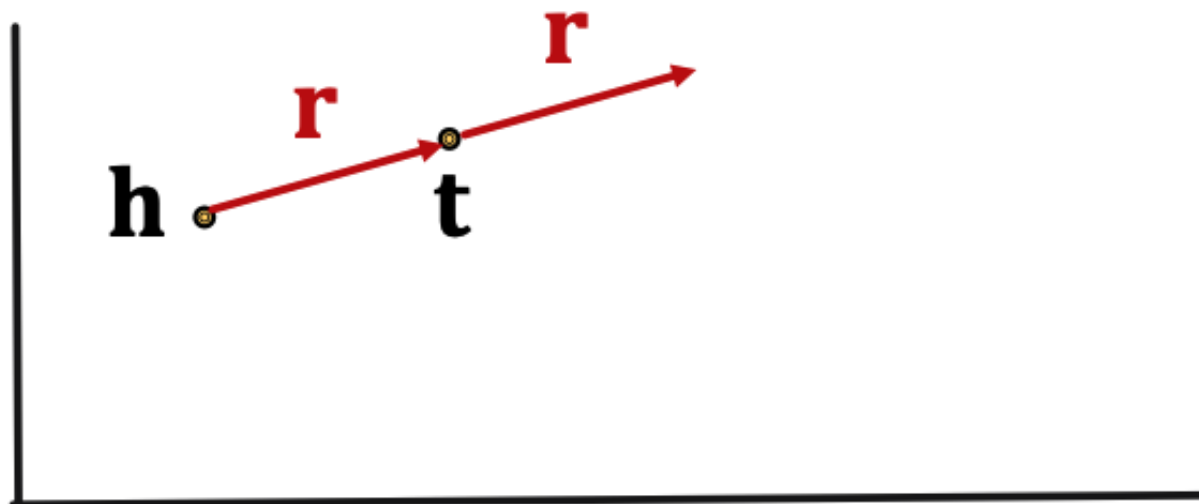
13: **end loop**

Contrastive loss: Favors lower distance (or higher score) for valid triplets, high distance (or lower score) for corrupted ones



TransE antisymmetric relations

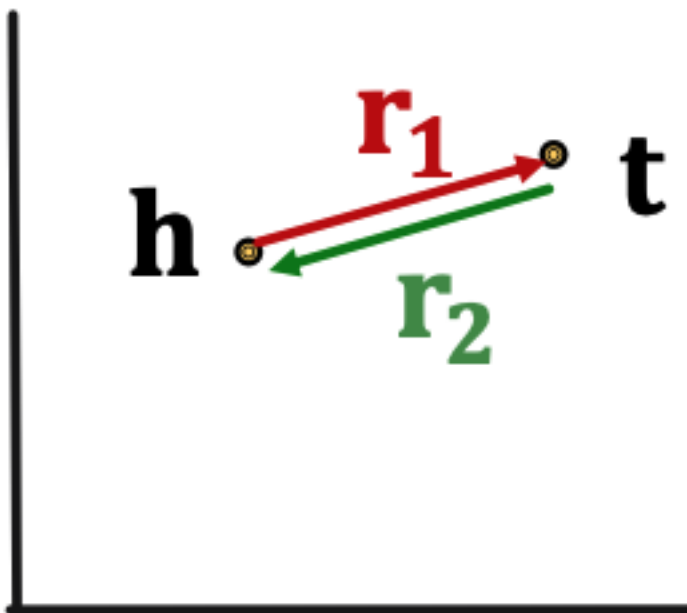
- $r(h, t) \Rightarrow !r(h, t)$
- Example: $\text{parent}(\text{Joe}, \text{Linda})$ then $!\text{parent}(\text{Linda}, \text{Joe})$
- TransE **can** model **antisymmetric** relations
 - $\mathbf{h} + \mathbf{r} = \mathbf{t}$ (by design), but $\mathbf{t} + \mathbf{r} \neq \mathbf{h}$





TransE inverse relations

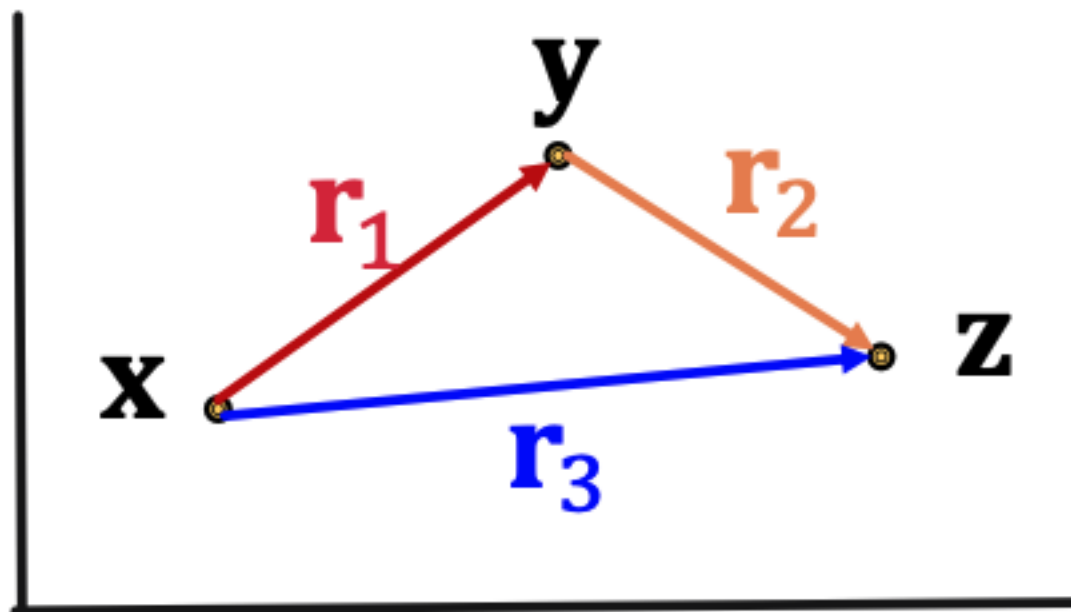
- $r_1(h, t) \Rightarrow r_2(t, h)$
- Example: (Mentor, mentee)
- TransE **can** model inverse relations
 - $\mathbf{h} + \mathbf{r}_2 = \mathbf{t}$ and $\mathbf{t} + \mathbf{r}_1 = \mathbf{h}$ by setting $\mathbf{r}_1 = -\mathbf{r}_2$





TransE composition relations

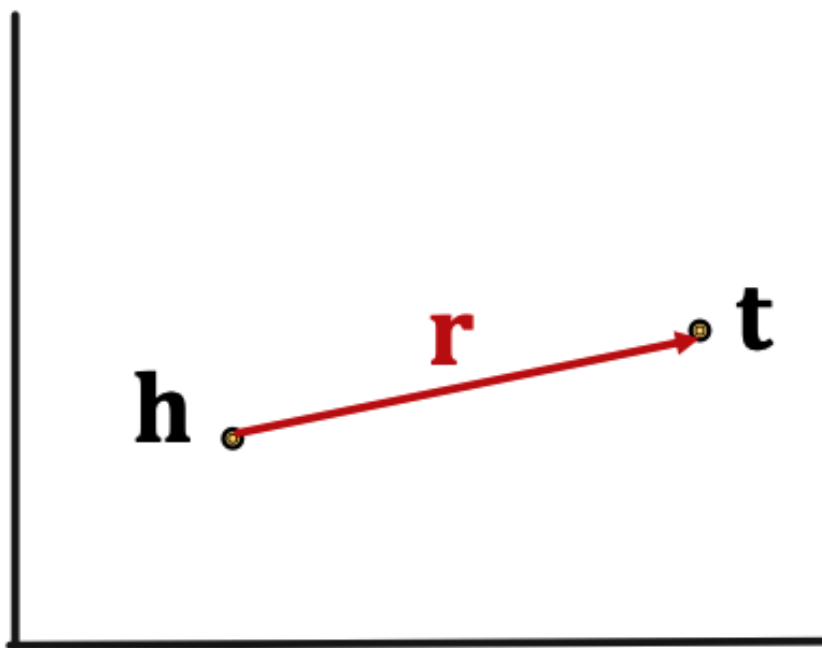
- $r_1(x, y) \wedge r_2(y, z) \Rightarrow r_3(x, z)$
- Example: parent (Joe, Linda) $\wedge$ father (Linda, Steve) $\Rightarrow$ grandfather (Joe, Steve) (the father of my parent is my grandfather)
- TransE **can** model **composition** relations by setting $r_3 = r_1 + r_2$





TransE symmetric relations

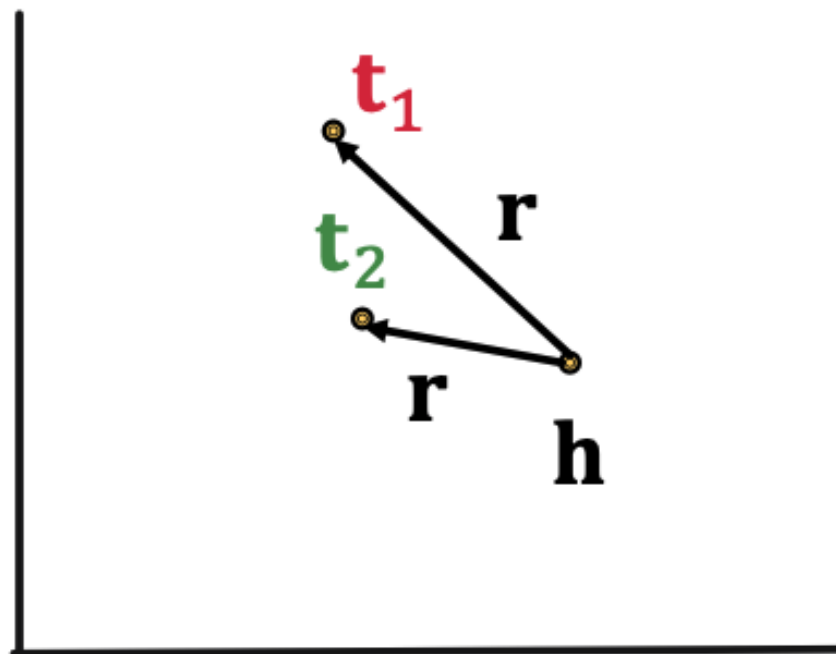
- $r(h, t) \Rightarrow r(h, t)$
- Example: family
- TransE **cannot** model **symmetric** relations
 - Requires $\mathbf{h} + \mathbf{r} = \mathbf{t}$ and $\mathbf{t} + \mathbf{r} = \mathbf{h}$ which can only hold if $\mathbf{r} = \mathbf{0}$ and $\mathbf{h} = \mathbf{t}$





TransE 1-to-N relations

- Consider (h, r, t_1) and (h, r, t_2) both exist
- Example: t_1 and t_2 are enrolled in ML4G
- TransE **cannot** model **both** relations assuming $t_1 \neq t_2$
 - As t_1 and t_2 are different entities they should have different embeddings





KG Completion Models

So far, we have

Model	Score	Embedding	Sym.	Antisym	Inv	Compos	1-to-N
TransE	$-\ \mathbf{h} + \mathbf{r} - \mathbf{t}\ $	$\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{R}^k$	✗	✓	✓	✓	✗



TransR method





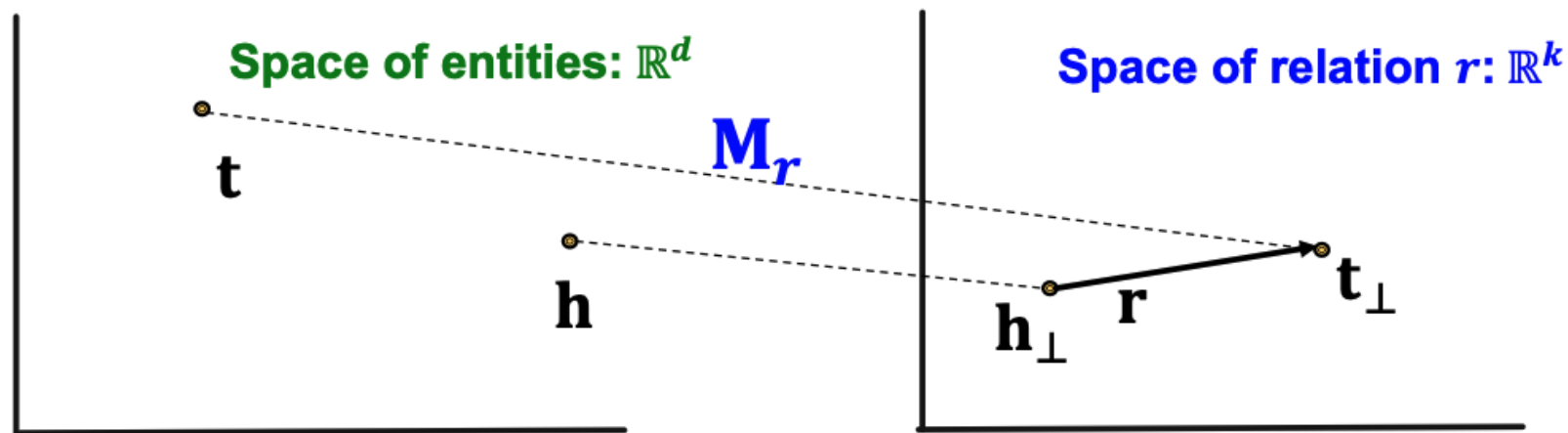
TransR

- **TransE** conceptually models the translation of any relation in the same latent space
- **TransR** models **entities** as vectors in an entity space $\mathbb{R}^d$, and models **each relation** in its own relation space, $r \in \mathbb{R}^k$, using a projection matrix $M_r \in \mathbb{R}^{k \times d}$
- For any pair of relations r_1 and r_2 , we can have $\mathbf{M}_{r_1} \neq \mathbf{M}_{r_2}$
 - We'll see that this gives us more flexibility than TransE



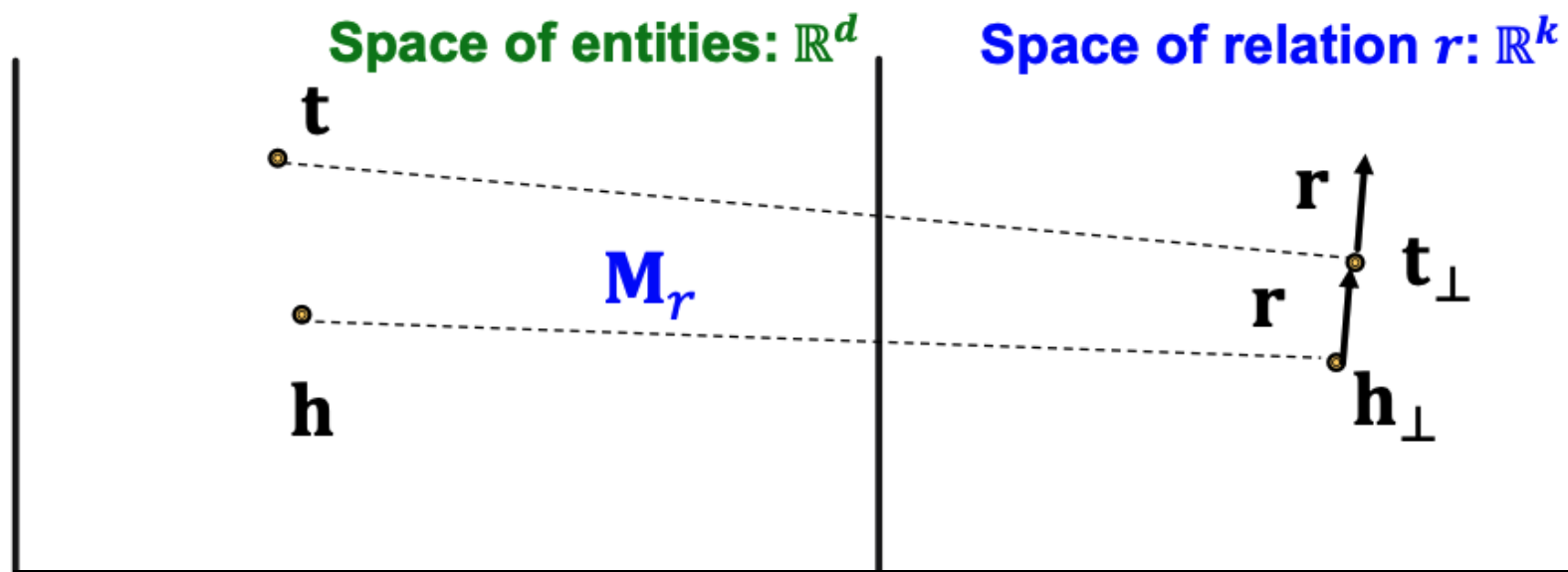
TransR

- TransR models **entities** as vectors in an entity space $\mathbb{R}^d$, and models **each relation** in its own relation space, $r \in \mathbb{R}^k$, using a projection matrix $M_r \in \mathbb{R}^{k \times d}$
- For a given $\mathbf{h}, \mathbf{r}, \mathbf{t}$, we set $\mathbf{h}_\perp = \mathbf{M}_r \mathbf{h}$ and $\mathbf{t}_\perp = \mathbf{M}_r \mathbf{t}$ to project $\mathbf{h}$ and $\mathbf{t}$ the entity space $\mathbb{R}^d$ to the relation space $\mathbb{R}^k$
- Score function: $f_r(h, t) = -\|\mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\|$



TransR antisymmetric relations

- $r(h, t) \Rightarrow !r(h, t)$
- Example: $\text{parent}(\text{Joe}, \text{Linda})$ then $!\text{parent}(\text{Linda}, \text{Joe})$
- TransR **can** model **antisymmetric** relations
 - $\mathbf{r} \neq \mathbf{0}$ and $\mathbf{M}_r \mathbf{h} + \mathbf{r} = \mathbf{M}_r \mathbf{t} = \mathbf{t}_\perp$ which implies $\mathbf{M}_r \mathbf{t} + \mathbf{r} \neq \mathbf{M}_r \mathbf{h} = \mathbf{h}_\perp$



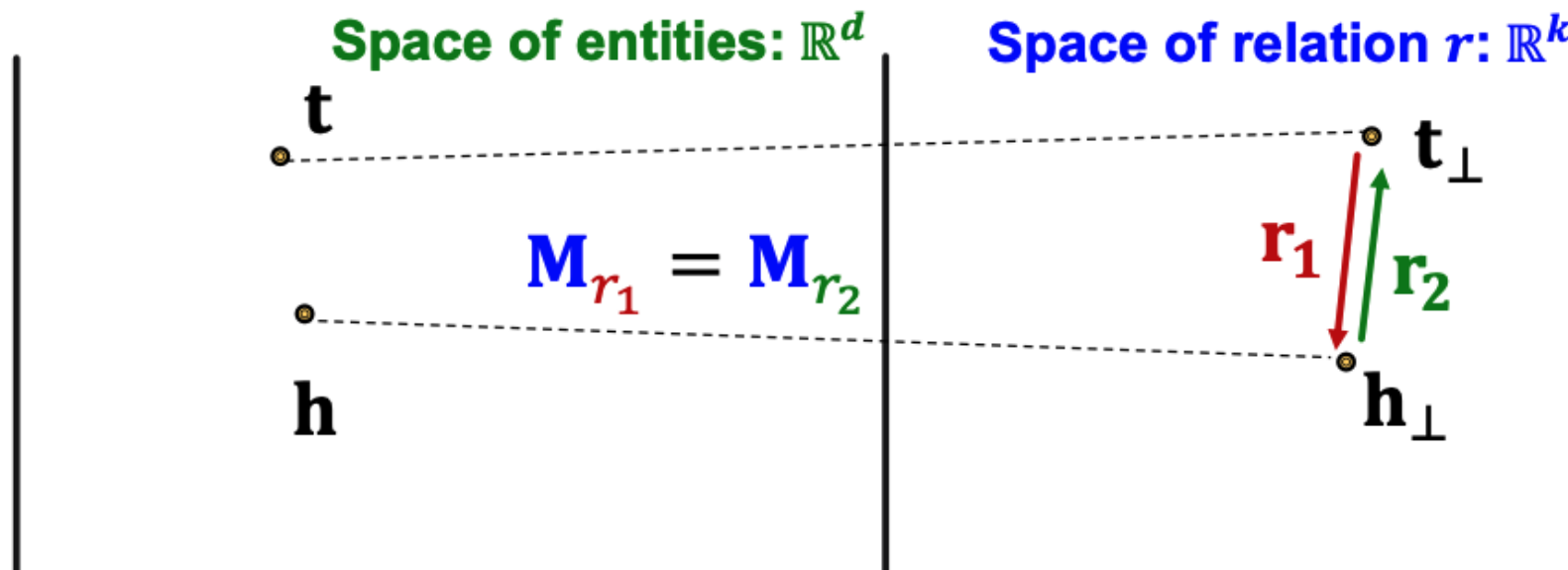


TransR inverse relations

- $r_1(h, t) \Rightarrow r_2(t, h)$
- Example: (Mentor, mentee)
- TransR **can** model inverse relations

$$\mathbf{r}_1 = -\mathbf{r}_2$$

Then $\mathbf{M}_{r_1} \mathbf{t} + \mathbf{r}_1 = \mathbf{M}_{r_1} \mathbf{h}$ and $\mathbf{M}_{r_2} \mathbf{h} + \mathbf{r}_2 = \mathbf{M}_{r_2} \mathbf{t}$

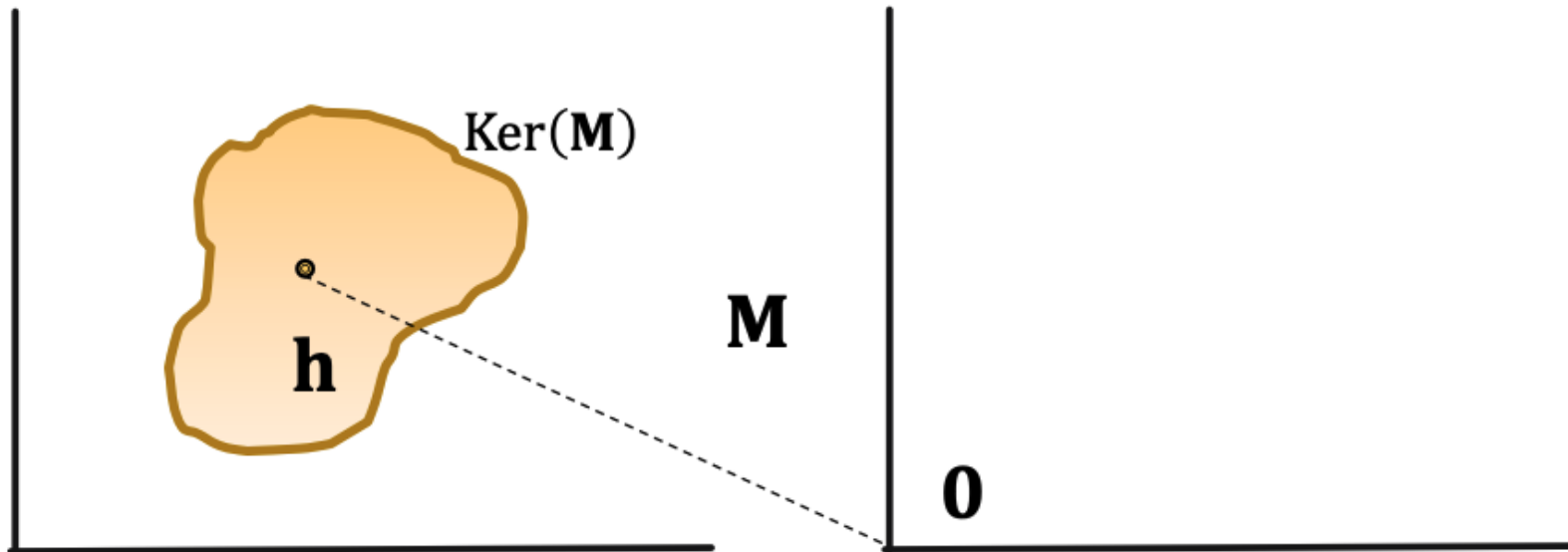




TransR and matrix null space

- Null space $N(\mathbf{M})$ (aka kernel space) of a matrix, $\mathbf{M}$, is the set of vectors that are mapped to the null ($\mathbf{0}$) vector
- If $\mathbf{h} \in N(\mathbf{M})$ then $\mathbf{M}\mathbf{h} = \mathbf{0}$

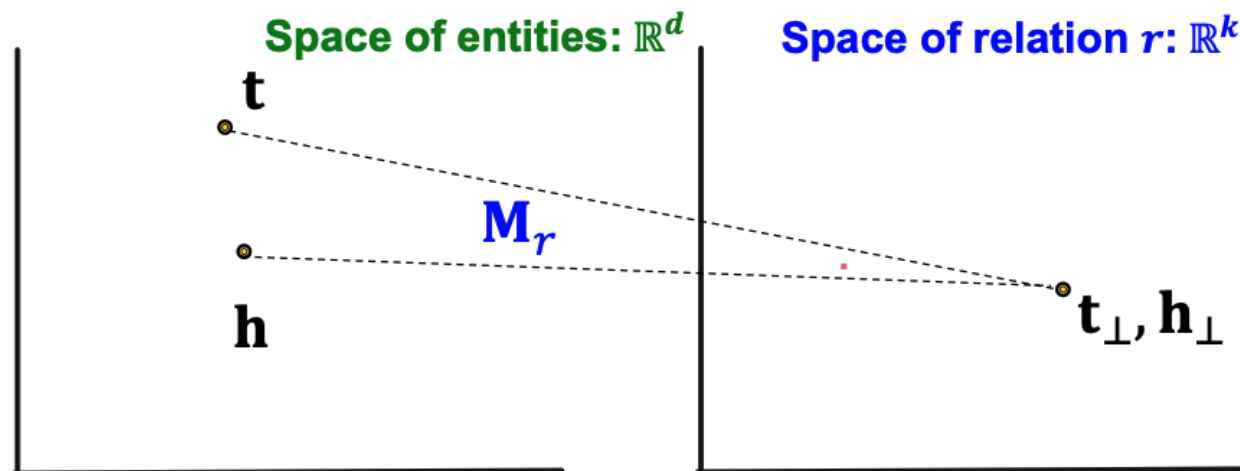
$$\mathbf{h} \in \text{Ker}(\mathbf{M}), \text{ then } \mathbf{M} \cdot \mathbf{h} = \mathbf{0}$$





TransR symmetric relations

- $r(h, t) \Rightarrow r(t, h)$
- Example: family
- TransR can model symmetric relations
 - Set $\mathbf{r} = \mathbf{0}$ and $\mathbf{h}_\perp = \mathbf{M}_r \mathbf{h} = \mathbf{M}_r \mathbf{t} = \mathbf{t}_\perp$
- Requires $\mathbf{M}_r \mathbf{h} - \mathbf{M}_r \mathbf{t} = \mathbf{0}$ Implies $\mathbf{h} - \mathbf{t}$ is in the null space, $N(\mathbf{M}_r)$

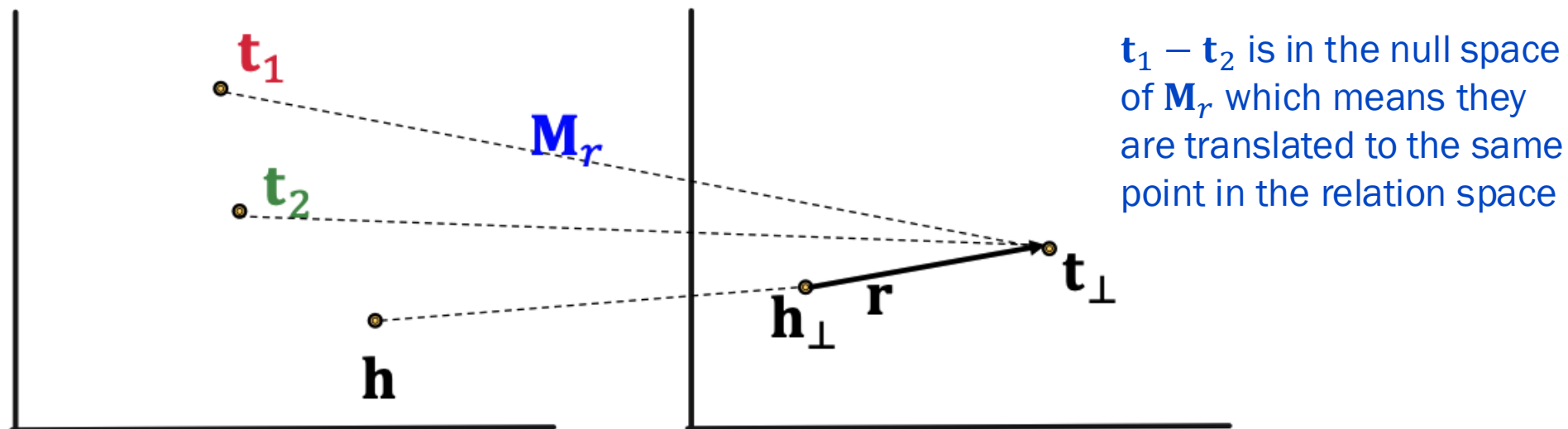




TransR 1-to-N relations

- Consider (h, r, t_1) and (h, r, t_2) both exist
- Example: t_1 and t_2 are enrolled in ML4G
- TransR **can** model **both** relations. Indeed, it can model any 1-to-N relation if we can find a matrix $\mathbf{M}_r$ such that for a given h , and for all entities t_i, t_j such that (h, r, t_i) and (h, r, t_j) exist, then $\mathbf{t}_i - \mathbf{t}_j$ is in the null space $N(\mathbf{M}_r)$ so that:

$$\mathbf{M}_r \mathbf{t}_i = \mathbf{M}_r \mathbf{t}_j$$





TransR composition relations

- $r_1(x, y) \wedge r_2(y, z) \Rightarrow r_3(x, z)$
- Example: $\text{parent}(\text{Joe}, \text{Linda}) \wedge \text{father}(\text{Linda}, \text{Steve}) \Rightarrow \text{grandfather}(\text{Joe}, \text{Steve})$ (the father of my parent is my grandfather)
- TransR **can** model **composition** relations
- Intuition: TransR models a triple with linear functions which are composable.
 - If $f(x)$ and $g(x)$ are linear, then the composition $f(g(x))$ is also linear:
Let $f(x) = ax + b$ and $g(x) = cx + d$ then $f(g(x)) = acx + (ad + b)$
- Incidentally, this is why non-linear activations are needed in deep learning, otherwise the multiple layers would just be equivalent to a linear function



TransR composition relations

- $r_1(x, y) \wedge r_2(y, z) \Rightarrow r_3(x, z)$
- Assume $\mathbf{M}_{r_1} \mathbf{g}_1 = \mathbf{r}_1$ and $\mathbf{M}_{r_1} \mathbf{g}_2 = \mathbf{r}_2$

$\mathbf{g}_1$ is a vector subspace of the entity embedding space.
 $\mathbf{M}_{r_1} \mathbf{g}_1 = \mathbf{r}_1$ implies for any vector $\mathbf{g} \in \mathbf{g}_1$ that $\mathbf{g}_1 - \mathbf{r}_1 \in N(\mathbf{M}_{r_1})$. The same is true for $\mathbf{g}_2$.

- If $r_1(x, y)$ exists $\Rightarrow \mathbf{M}_{r_1} \mathbf{x} + \mathbf{r}_1 = \mathbf{M}_{r_1} \mathbf{y} \Rightarrow \mathbf{M}_{r_1} (\mathbf{y} - \mathbf{x}) = \mathbf{r}_1$

$$\mathbf{y} - \mathbf{x} \in \mathbf{g}_1 + N(\mathbf{M}_{r_1}) \Rightarrow \mathbf{y} \in \mathbf{x} + \mathbf{g}_1 + N(\mathbf{M}_{r_1})$$

- If $r_2(y, z)$ exists $\Rightarrow \mathbf{M}_{r_2} \mathbf{y} + \mathbf{r}_2 = \mathbf{M}_{r_2} \mathbf{z} \Rightarrow \mathbf{M}_{r_2} (\mathbf{z} - \mathbf{y}) = \mathbf{r}_2$

$$\mathbf{z} - \mathbf{y} \in \mathbf{g}_2 + N(\mathbf{M}_{r_2}) \Rightarrow \mathbf{z} \in \mathbf{y} + \mathbf{g}_2 + N(\mathbf{M}_{r_2})$$

- Then, we have

$$\mathbf{z} \in \mathbf{x} + \mathbf{g}_1 + \mathbf{g}_2 + N(\mathbf{M}_{r_1}) + N(\mathbf{M}_{r_2})$$



TransR composition relations

- $r_1(x, y) \wedge r_2(y, z) \implies r_3(x, z)$
- From the previous slide:

$\mathbf{g}_1$ is a vector subspace of the entity embedding space.
 $\mathbf{M}_{r_1}\mathbf{g}_1 = \mathbf{r}_1$ implies for any vector $\mathbf{g} \in \mathbf{g}_1$ that $\mathbf{g}_1 - \mathbf{r}_1 \in N(\mathbf{M}_{r_1})$. The same is true for $\mathbf{g}_2$.

$$\mathbf{z} \in \mathbf{x} + \mathbf{g}_1 + \mathbf{g}_2 + N(\mathbf{M}_{r_1}) + N(\mathbf{M}_{r_2})$$

- We now construct $\mathbf{M}_{r_3}$ such that

$$N(\mathbf{M}_{r_3}) = N(\mathbf{M}_{r_1}) + N(\mathbf{M}_{r_2})$$

and set

$$\mathbf{r}_3 = \mathbf{M}_{r_3}(\mathbf{g}_1 + \mathbf{g}_2)$$

$\mathbf{g}_1 + \mathbf{g}_2 - \mathbf{r}_3$ is a vector subspace of the entity embedding space and all vectors in this subspace are in the null space $N(\mathbf{M}_{r_3})$

- Finally, we have

$$\mathbf{M}_{r_3}(\mathbf{z}) \in \mathbf{M}_{r_3}\mathbf{x} + \mathbf{M}_{r_3}(\mathbf{g}_1 + \mathbf{g}_2) + \boxed{\mathbf{M}_{r_3}(N(\mathbf{M}_{r_1}) + N(\mathbf{M}_{r_2}))}$$

$$\mathbf{M}_{r_3}(\mathbf{z}) \in \mathbf{M}_{r_3}\mathbf{x} + \mathbf{M}_{r_3}(\mathbf{g}_1 + \mathbf{g}_2) = \mathbf{M}_{r_3}\mathbf{x} + \mathbf{r}_3$$

necessarily a null vector

$$\mathbf{M}_{r_3}(\mathbf{z}) = \mathbf{M}_{r_3}(\mathbf{x}) + \mathbf{r}_3$$



KG Completion Models

Model	Score	Embedding	Sym.	Antisym	Inv	Compos	1-to-N
TransE	$-\ \mathbf{h} + \mathbf{r} - \mathbf{t}\ $	$\mathbf{h}, \mathbf{r}, \mathbf{t} \in \mathbb{R}^k$	✗	✓	✓	✓	✗
TransR	$-\ \mathbf{h}_\perp + \mathbf{r} - \mathbf{t}_\perp\ $	$\mathbf{h}, \mathbf{t} \in \mathbb{R}^d$ $\mathbf{r} \in \mathbb{R}^k$ $\mathbf{M}_r \in \mathbb{R}^{k \times d}$	✓	✓	✓	✓	✓



KG completion in practice

- There are numerous other methods for KG completion
- Different KGs may have drastically different relation patterns
- There is not a general method (yet) that works for all KGs
- Try TransE for a quick analysis
- Then use more complex (TransR, ComplEX, RotateE, ...)

